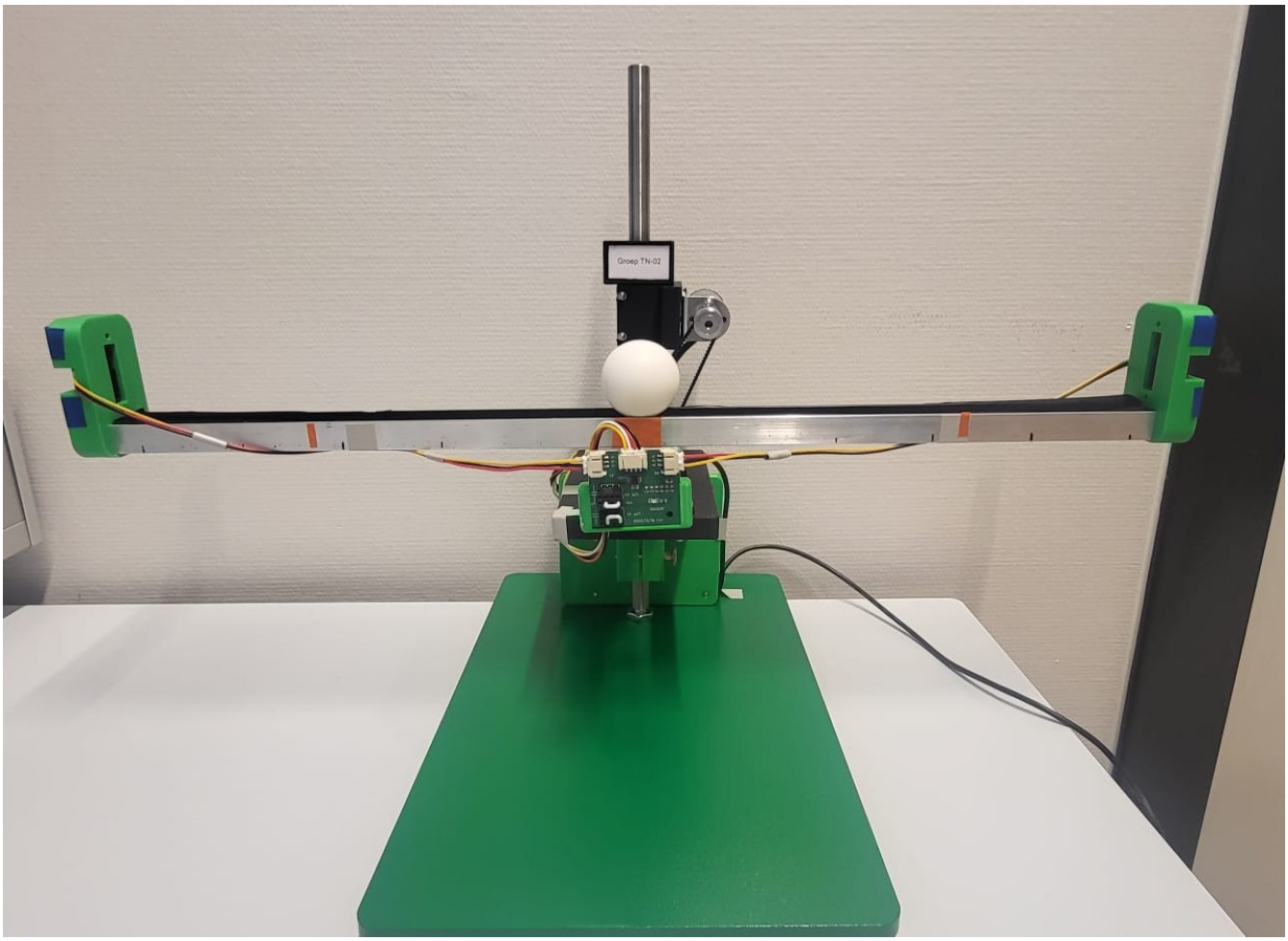
Modelling and designing of self-balancing seesaw

J.M. de Vries & L.J.M. Taken

November 2025



Abstract

In this report the self balancing seesaw is designed and modelled using one PID and one P controller. The final design balanced a ball on the seesaw in 10 seconds with an average overshoot of 5 cm.

Contents

1 Introduction **3**

2 Program of requirements **4**
2.1 Objective . . . 4
2.2 Constraints . . . 4
2.3 Requirements . . . 4
2.4 Challenges . . . 4

3 Functional analysis **5**
3.1 Conceptual design . . . 5

4 Detailed design **6**

5 Testing to determine system parameters **8**
5.1 Determining α and R . . . 8
5.2 Determining L . . . 8
5.3 Determining J . . . 8
5.4 Determining B . . . 9

6 Building and testing **9**
6.1 Testing . . . 9
6.2 Test results . . . 9

7 Evaluation and discussion **11**
7.1 Comparison model and measurements . . . 12
7.2 Improvements . . . 12

8 Conclusion **12**

A Calibration distance sensors **14**

B Deriving of differential equation 5 **16**

C Deriving of differential equation 2 **17**

D Code **18**

1 Introduction

In this report, the process of developing a self balancing seesaw is described. First, the objective, requirements and challenges are presented in the program of requirements to get a better understanding of the goal of the system. Then, the system is modeled calculating the differential equations for the separate subsystems. These are then used to test the stability of the system and to calculate the theoretical margins for the controllers. Then, the system parameters are determined and the whole system is tested. Lastly, the performance of the system is discussed and the theoretical model and the final system are compared. From this, conclusions will be made on the strengths and weaknesses of the design.

After reading this report, the reader will get a better understanding of how a self balancing seesaw is designed using controllers. They can also use the presented information to design a upgraded version of the seesaw based of the evaluation of the presented design.

2 Program of requirements

In this section the objective of the project is explained together with the limitations of the project, the requirements of the system and the challenges that might occur during the process.

2.1 Objective

The objective of this project is to balance a ball on a beam. Specifically, a ping pong ball is placed somewhere on a 53 cm long beam and the task is then to get the ball stabilized in the middle of the beam. This task needs to be done within a 20 mm overshoot of the target and in a time span of 5 seconds or less. The maximum steady state error is 10 mm. The input of the system is an input voltage U_s that generates a current i that gives a torque to the beam. The angle of the beam then ensures that the ball gets a velocity and a displacement as output. The displacement is then carried back into the system as feedback.

One of the basic functions is measuring the distance of the ball with respect to the distance sensor. Another function is to rotate the beam using the pulses of the rotary encoder to a specific angle θ. These specific angles are managed with a controller that uses the output angle as feedback to get the beam to the desired angle. An additional function is a controller that controls the position of the ball. This controller takes the measurements of the distance sensors to adjust the seesaw and get the ball to the wanted position.

One of the performance indicators are the measurements of the distance sensors that show the position of the ball. Another indicator is the settling time seesaw, so how fast the seesaw gets the ball to the desired position. An equally important indicator is the rise time. This is the time the system takes to response to a disturbance of the system. The amount of overshoot and the steady-state error also indicates the performance of the system. The last performance indicator of the system is the consistency of the results.

It is desired to have a fast settling and rise time. The error and overshoot also need to be low. The system also needs to be consistent.

2.2 Constraints

There are multiple constrains while constructing a model and prototype for the seesaw. Mechanical constraints of the system are the limitations to the motor and the sensors. Specifically the motor speed, motor response time, motor torque, sensor accuracy and sensor response time. There are also constraints of the used control systems. Each control strategy (P,PD and PID) has advantages and disadvantages. A choice of one of these will influence the complexity of the system, but also the amount of the steady-state error, amount of overshoot and resistance of noise[1]. Another type of constraints are the environmental constraints. This means the existence of light variations, vibrations and difference surfaces. The systems also needs to be safe to use and needs to be portable. This means the the system can't be bigger than 1 m3 and not heavier than 40 kg. Last but not least, the last constraints are the time in which the seesaw needs to be build (8 weeks),the manpower available in the meantime (2 people) and the budget available for the project (no budget).

2.3 Requirements

In order to complete the objective stated in paragraph 2.1, the system needs to have the following requirements. First of all, the system needs to be able to detect the position of the ball on the beam. This can be done by using the distance sensors on either side of the beam and relating output voltage of those sensors to the position of the ball. This calibrating process needs to be done for both sensors and then be related to each other. The second requirement is that the motor needs to go to a specific angle to move the ball. This is done by using a PID controller to link the position of the ball to the angle the beam needs to have. The beam then needs to move to the required angle using the motor. These two requirements will cause the system to complete the objective of stabilizing the ball in the middle of the beam. Lastly, the system is also required to be robust and it should be able to be used multiple times in a row. This means that a way to start and stop needs to be implemented.

2.4 Challenges

One of the challenges that may occur during the designing process is the error in the measurement of the distance of the ball with the distance sensors. This can be dealt with by measuring multiple times, fitting the graph accordingly and combining the measurements of both distance sensors for the best accuracy. Errors in the measurements of the variables in the transfer function can also form a challenges due to differences with the model and the system. This can cause wrong PID values. Assumptions and simplifications in the model can also have the same effect. You can correct these wrong PID values by testing the design and changing the PID values according to the results. An equally important challenge that might occur is that the system is not robust enough. This can be caused if the variables in the system are dependent on one type of environment. You can solve this by depending the starting values on the environment and calibrating at the start of every try. This makes sure that the system is robust under

multiple circumstances. Lastly, different updating times for the sensor can cause wrong outputs for the system. This can be accounted for in the code of the system.

3 Functional analysis

3.1 Conceptual design

3.1.1 Model and free body diagram

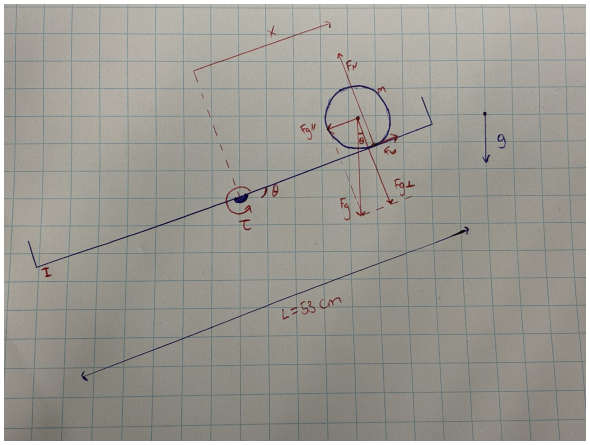


Figure 1: Free body diagram

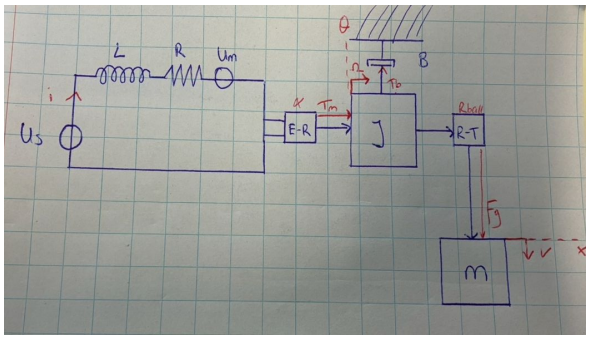


Figure 2: Lumped element model

There are 3 subsystems in this model, namely the DC motor, the beam and the ball. The beam has current i as state variable because of the inductor in the motor. There is also a resistor in the motor. A back-emf is generated as well, this is Um in the model. The input is Us. The parameters of the system are displayed in the table below. These tests can be found in the chapter: "Testing to determine system parameters".

Table 1: Table with system parameters

Dimension	B [Nms/rad]	α [Vs/rad]	L [H]	J [kg/m2]	R [Ω]
Value	0.17	0.36	0.008	0.0068	5.67

The DC motor is coupled to the beam with rotational inertia J. The motor delivers a torque Tm to the beam. The beam also experiences rotational friction, which produces torque Tb. The angular velocity of the beam is Ω and the angle of the beam is θ.

The ball is balanced on the beam. The angle of the beam influences the force of gravity parallel to the beam. We assume that the ball is rolling without slipping, so the inertia of the ball needs to be accounted for. The relation is as followed:

$$F_{res} = mgsin(\theta) - \frac{2}{3}ma \tag{1}$$

There are 5 differential equations needed to describe this system:

$$L\frac{di}{dt} = Us - iR - \alpha\Omega \tag{2}$$

$$J\frac{d\Omega}{dt} = \alpha i - B\Omega \tag{3}$$

$$\frac{d\theta}{dt} = \Omega \tag{4}$$

$$m\frac{dv}{dt} = \frac{3}{5}mgsin(\theta) \tag{5}$$

$$\frac{dx}{dt} = v \tag{6}$$

Here L is the inductor, R is the resistor, i is the current, Us is the input voltage, α is the coupling coefficient of the motor, Ω is the angular velocity of the beam, J is the inertia of (everything rotating connected to) the beam, θ is the angle of the beam, m is the mass of the ball, v is the velocity of the ball, g is the gravitational constant and x is the position of the ball. All these constants are also introduced in the free body diagram and are in SI-units. Equation 2 and 5 are derived in appendix B and C.

Linearization is needed in the equation for the gravity working on the ball. This force is proportional to the sine of the angle θ, but for small angles, we can use the small angle approximation:

$$sin(\theta) = \theta \tag{7}$$

Then, equation 5 becomes:

$$m\frac{dv}{dt} = \frac{3}{5}mg\theta \tag{8}$$

The position of the ball is measured with IR distance sensors. These sensors generate a voltage when a distance is measured. This relation is not linear. The characterization of the sensors can be found in Appendix A.

The angle of the beam is measured by the rotary encoder. The encoder gives a certain amount of pulses for an angle. This was measured manually. If you divide the pulses by this amount of pulses that is a 360 degrees

rotation, you can determine the beam angle. The rotary encoder measured 1019 pulses per 360 degree turn.

3.1.2 Block diagram

The block diagram of the whole system is presented in figure 3.

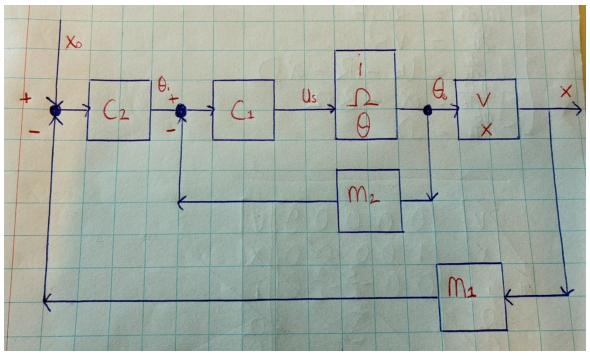


Figure 3: Block diagram

There are two feedback loops. One is to regulate the input voltage Us based on the output angle θ. This is the connection between the beam and the motor. The performance requirements of this subsystems are that it must generate enough torque to counteract friction and make the beam rotate. Another requirement is that it must respond fast to the input that is given. Lastly, the beam needs to consistently rotate to an angle with a given input voltage. This can be measured using the rotary encoder. The angle also cannot be more than 10 degrees due to the small angle approximation made before.

The other feedback loop controls the angle of the beam based on the output position x of the ball. This is the connection between the beam and the ball. One of the requirements is the measurement of the ball position with the distance sensors. This measurement can then be used to minimize overshoot, handle disturbance and get the ball to the desired position. All these actions are also requirements of this subsystem.

This is in general the conceptual design of the seesaw. This concept has five state variables and two controllers. In the next chapters, the detailed design is explained and the system variables are derived during testing.

4 Detailed design

In this paragraph, the detailed design is shown and explained. This is done by calculating the transfer functions of the model. From this, the PID values are determined.

The transfer function is determined by first getting the transfer function of the smaller loop inside. Then, you take the the transfer function of the whole system. This turned out to be:

$$T_s(s) = \frac{3g\alpha C1C2}{5s^3((B+sJ)(R+Ls)+\alpha^2)+\alpha C1(5s^2+3gC2)} \tag{9}$$

To analyse stability, we will look at the individual transfer functions of the systems. The transfer function of the motor on the beam is:

$$T_1(s) = \frac{\alpha}{s((B+Js)(R+Ls)+\alpha^2)} \tag{10}$$

This is multiplied with C1 to create the first open loop transfer function. This function has a pole which is s=0. This means that the system is marginally stable. A C1 can be calculated for which this system exists in the stable range. This is done by making a root locus plot.

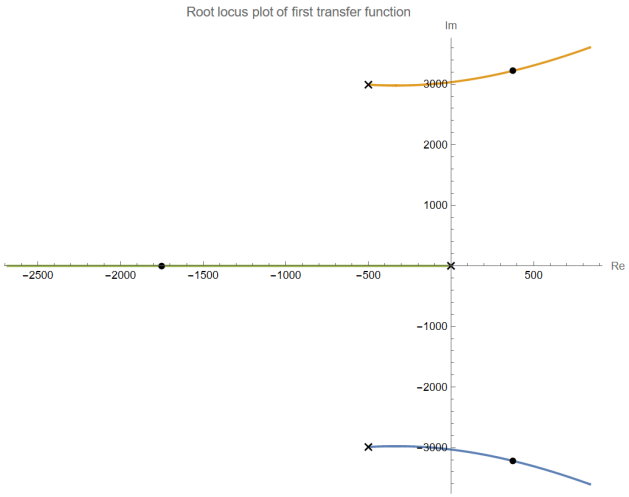


Figure 4: Root locus of first transfer function

As mentioned earlier, the system has a pole of 0. There are also 2 negative poles at C1 = 0. When C1 is increased, the poles shift towards larger real parts, until they become positive. The largest value of C1 for which the real parts are not yet positive is 25000. Therefore, C1 needs to be somewhere between 0 and 25000 to stabilize the system.

Another root locus plot is made of the transfer function of the ball. The transfer function of the beam on the ball is:

$$T_2(s) = \frac{3g}{5s^2} \tag{11}$$

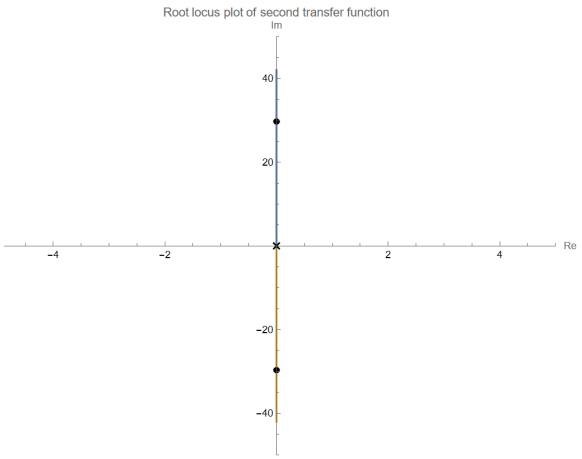


Figure 5: Root locus of second transfer function

It is shown in this root locus plot that the poles always have a real part of zero. This is logical if you look at the transfer function. The imaginary part gets larger with a larger C2, but since the real part is always 0, the system is always stable for all values of C2.

These transfer functions are combined using block algebra. The transfer function of the complete system is:

$$Tol_(s) = \frac{3C2C1\alpha g}{5s^3((B+sJ)(R+sL)+\alpha^2)+\alpha C1(5s^2+3gC2)} \quad (12)$$

Instead of filling in constants for C1 and C2, they are taken to be PID controllers. For the first simulation, k_p is 1, τ_d is 0 and τ_i is 99 (large value). This simulation results in the following step response of the entire system:

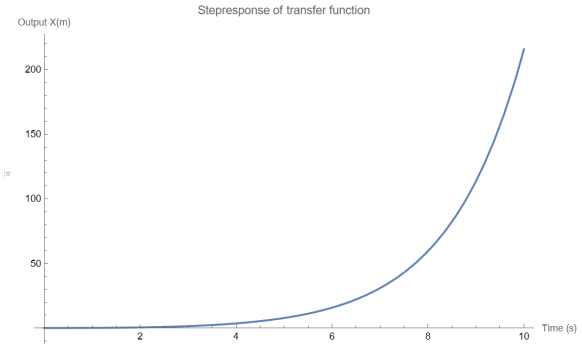


Figure 6: First step response simulation

This is of course not the desired response. The system increases exponentially with a step input, and it should stabilize at 1. Therefore, the parameters of k_p, τ_d and τ_i need to be determined as described in this reader [2].

The speed of the control depends on τ_i and τ_d. With τ_i, it is possible to compensate for a pole with a zero in the control function. If you compensate for the smallest pole, you get rid of the largest time constant of your system and you get a faster control. It also depends on the amount of overshoot that is allowed. The extra pole in zero reduces the steady state error to zero as well.

With τ_d, you can also compensate for a pole, but without adding an extra pole in zero. This means that the steady state error is not reduced with just a PD controller. You compensate again for the smallest pole, because that pole has the largest time constant and therefore reacts the slowest.

For both controllers, a PID control will be set-up. With the step response data of the model, starting values for k_p, τ_i and τ_d will be determined. Then, with the entire system in place, these values can be optimized depending on the performance of the system. This tuning process is done by experimenting with the PID values and comparing the different step responses. You can see this in figure 7.

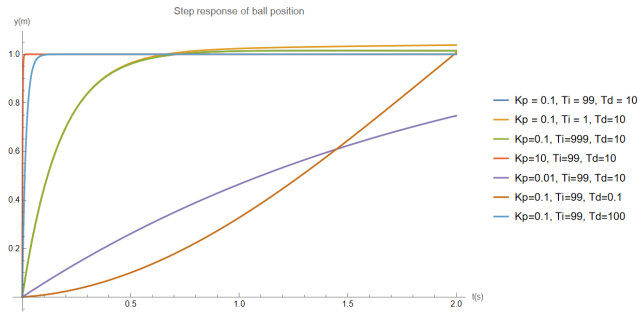


Figure 7: Example of empirical tuning for ball position

The optimal step response is one with a high rising time and a low overshoot. The upper blue line is with the optimal parameters of the control. A lower Td will cause the system to be unstable. Also, a higher Ti will cause the system to be slower. A lower Kp will cause the system to be way slower, while a higher Kp causes a faster system. You might conclude that Kp must be very high, but that can cause the system to be faster than the input of the sensors and the motor. This will disrupt the system.

The tuning process for the motor angle is shown in figure 8. Again, the optimal design is the upper blue line. The same requirements apply for this system, you want a fast response, but a low overshoot. A low Kp causes the system to be slow. A lower Td will cause the system to have a overshoot. Also, a lower Ti and a higher Ti will cause the system to be too fast or too slow.

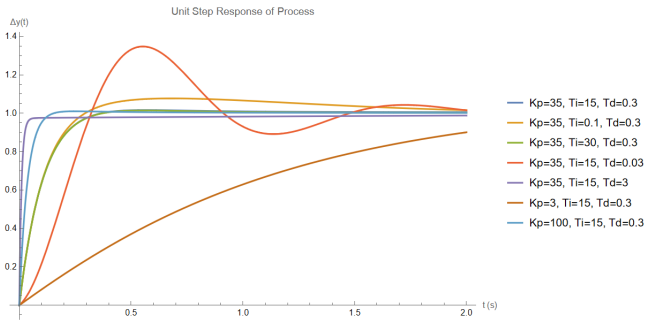


Figure 8: Example of empirical tuning for motor angle

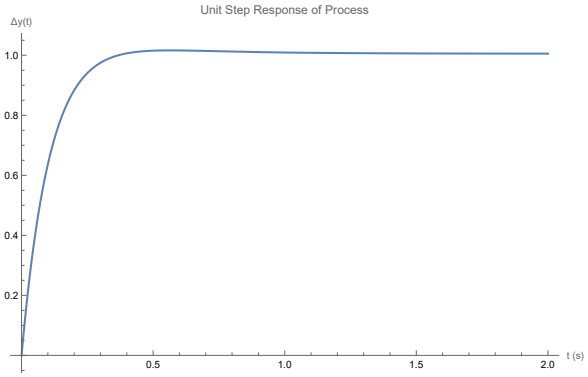


Figure 9: Step response of first transfer function (eq. 10)

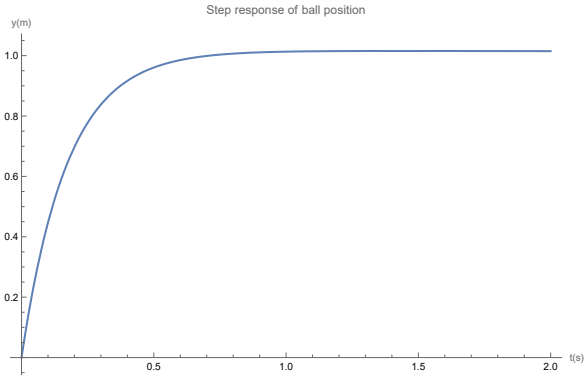


Figure 10: Step response of second transfer function (eq. 11)

In figure 9 and figure 10, the results of experimental tuning of the PID-values are shown again. For C1, k_p is 35, τ_i is 15 and τ_d is 0.3. For C2, k_p is 0.1, τ_i is 999 and τ_d is 10. This means that C1 needs to be a PID controller and C2 needs to be a PD controller.

These values are good enough as starting point for testing. The objective was to balance the ball at the middle of the beam in less than 5 seconds with an overshoot of 20 mm. The step response of our system shows that this is possible with these PID values. Of course, the response in real life will be different due to the fact that our model is simplified, but these values will be used to start with.

5 Testing to determine system parameters

There are a few system parameters that need to be determined in order to get an accurate model. Below, the different tests are preformed and explained.

5.1 Determining α and R

α is the coupling coefficient between the motor and the seesaw in V s/rad. R is the resistance of the motor in Ω. To determine R and alpha, you can use the specifications of the motor (see figure 11).

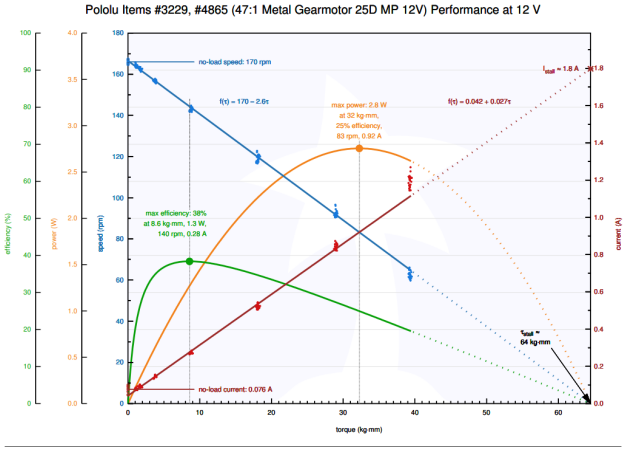


Figure 11: specifications of the motor

You can calculate α using the slope of the red line and the coupling equation 13. Using these, you find an α of $\alpha = 0.36Vs/rad$.

$$T = \alpha I \tag{13}$$

You can use the I_{stall} of 1.8 A to calculate R using Ohm's law. I_{stall} is at an input voltage of 12 V and an efficiency of 0.90. This is a voltage of 10.2 V. With this current and voltage, the resistance is $R = 5.67\Omega$.

5.2 Determining L

L is the inductance of the motor in H. This value is determined by putting a multimeter on the motor and reading the result. This gives a values of $L = 0.008H$.

5.3 Determining J

J is the inertia of the seesaw including the ball in kg/m^2. This parameter is determined by separating the seesaw in rectangles and determining the inertia of those rectangles with respect to the rotational axis. The rectangles are the three sides of the aluminum part of the seesaw

and the two holders of the sensors. The calculation is done with the parallel axis theorem.

$$J = \frac{1}{12}m(a^2 + b^2) + md^2 \tag{14}$$

In this case, a and b are the sides of the rectangle that are orthogonal to the rotational axis. Since the center of mass is not in the same point as the rotational axis, the factor md^2 is added with d the distance between the rotational axis and the center of mass. If you then add all the inertia's of the rectangles, you get an inertia of $J = 0.0068kg/m^2$. This is the inertia of the seesaw without the ball.

The inertia of the ball is depended on the distance of the ball with respect to the rotational axis. Since the ball is moving, the inertia changes. The maximum inertia of the ball is when the ball is on the edge of the seesaw. The inertia of the ball is then $J_{max,ball} = 0.00018kg/m^2$. This value is way smaller than the inertia of seesaw. Because of this, we neglect the inertia of the ball and only take the inertia of the seesaw.

5.4 Determining B

B is the friction of the seesaw in Nms/rad. B is determined using formula 15.

$$T = B\Omega \tag{15}$$

This equation states that in a T,Ω graph, the slope of the graph is B. Ω can be found by using the rotary encoder. T is calculates using the coupling formula 13. I is calculated using the differential equation 2. This gives the equation 16

$$I = \frac{Us - \alpha\Omega}{R} \tag{16}$$

With Us the input voltage of the motor. If you then plot Ω over T, you get the plot shown in figure 12.

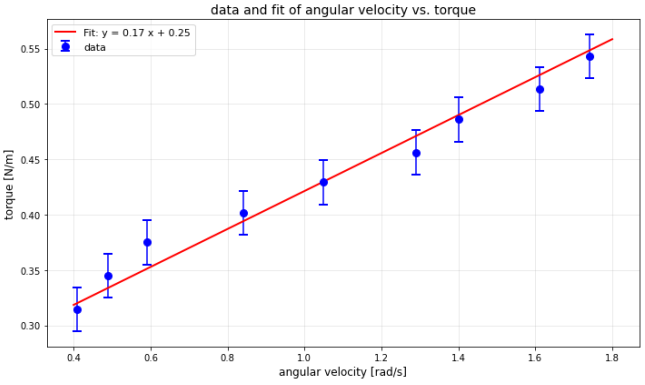


Figure 12: Torque vs. angular velocity

You can get B out of the slope of the figure, which is $B = 0.17Nms/rad$.

6 Building and testing

6.1 Testing

The model derived in the previous section and an example script for a PID controller (see appendix D) were used to test the seesaw. Firstly, the controller for the distance of the ball was tested. The conclusion was quickly drawn that the PID values of the model made the system unstable. This has to do with the fact that the PID parameters could be defined differently in the module that was used to program the controller. Since a conversion factor was not known, the values from the model could not be implemented directly.

At first, the PID values were optimized using trial and error. When the system did not improve anymore, the positions as a function of measurements were plotted.

6.2 Test results

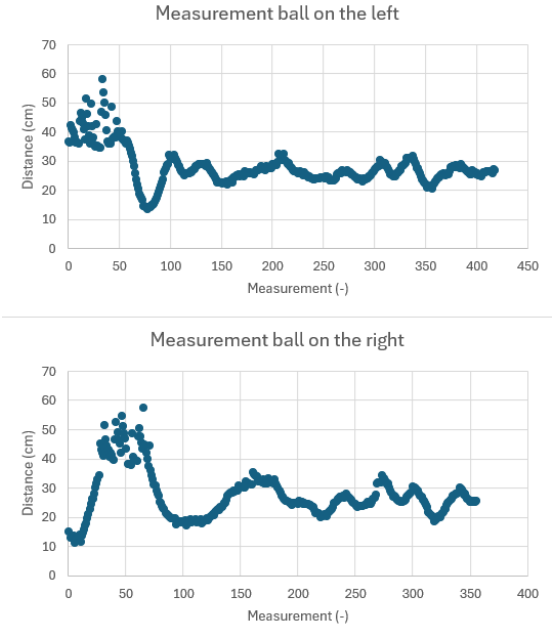


Figure 13: Measurement with initial code

This measurement was performed using a PID controller for the motor control and a P controller for the angle control. The PID values for the position control were a K_p of 0.4, a K_i of 0.3 and a K_d of 0.5. The K_p value of the angle control was 0.1.

The position of the ball was measured for around 20 seconds. The middle of the beam is around 26.5 cm. The goal is to have a maximum overshoot of 20 mm with a maximum of 10 mm steady state error. The overshoot is too large with these values. A logical approach would be to increase K_d and to decrease K_p to try and minimize the oscillation. K_p was changed from 0.4 to 0.3

and K_d was changed from 0.5 to 0.6. This was added into the code and new measurements were performed.

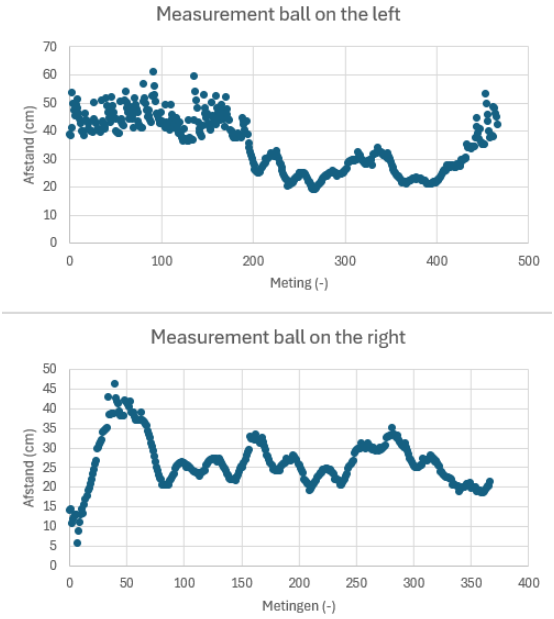


Figure 14: Measurement with changed K_p and K_d

These new PD values yielded a worse result. Therefore, the values were changed back into the original ones. Another cause for the oscillation was found. The sample time of the distance controller was initially quite small (0.001 s). If the rest of the system is slower, a lag will be introduced, causing the system to react wrongly. The sample time was increased to 0.005 seconds and new measurements were performed.

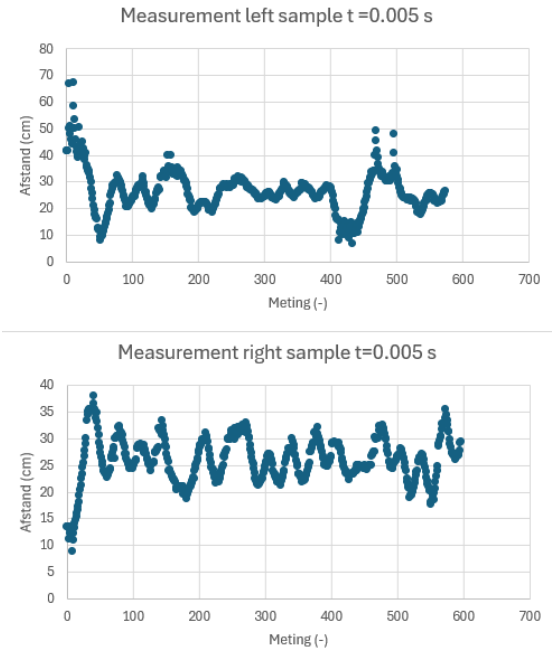


Figure 15: Measurement with changed sample time

This improved the system, so an even higher sample time was tried. Unfortunately, this did not yield better results. Then, a higher throttle was implemented in the system. The idea behind this strategy is that the system would react more strongly to the position of the ball, causing the ball to obtain the equilibrium position even faster and to minimize the overshoot. A line was plotted to visualize the setpoint, which was set at 26 cm.

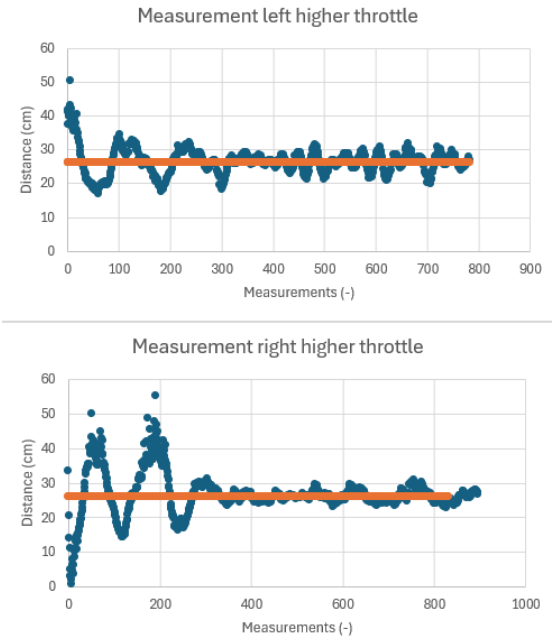


Figure 16: Measurement with changed throttle

Different throttles were tried to optimize the response. Eventually, the optimum throttle was found. The system responded as can be seen in figure 13. A line was added to visualize the desired position. This results already comes close to the objective described in chapter 2. With this improved code, the PID values were all tested again. The system deteriorated visually when K_p and K_d were either increased or decreased. The system seemed to improve when a lower K_i was taken. Therefore, new measurements were performed.

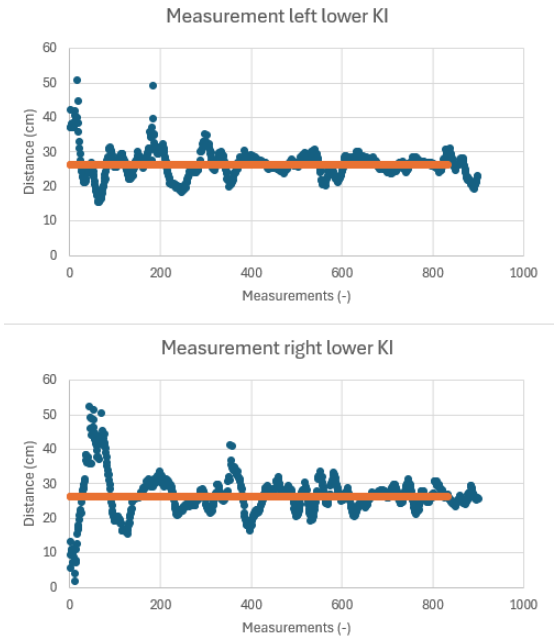


Figure 17: Measurement with changed K_i

Even though this change seemed to improve the system visually, it is clear that this change of K_i did not actually improve the system. However, it is possible that the distance sensors are somewhat unreliable. This could also be the cause as to why the system balances the ball more steadily when the ball starts on the left side of the beam. Taking all of this into account, the decision was made to change the K_i value back to 0.3. The test was performed with a K_i of 0.2. This is also the final system that was shown at the demonstration.

This test concludes that the values in the beginning are the most efficient. So, for the motor a Kp of 0.4, a Ki of 0.3 and a Kd of 0.5 and for the angle a Kp of 0.1

7 Evaluation and discussion

The objective was to stabilize the ball in the middle of the beam within a 20 mm steady state error of the target and in a timespan of 5 seconds or less. This objective has not been reached. It takes this system around 10 seconds to get to a stable situation, but this stable situation also has a larger overshoot than 20 mm. The set-up is safe, portable and everything is carefully aligned. Also, the results are consistent.

Final measurements will be performed to determine the settling time, the rise time, the overshoot and the steady state error. 2 sets of measurements will be performed, one with the ball 10 cm from the left end and one with the ball 10 cm from the right side. Then, the position of the ball will be measured as a function of time. These measurements will be done some more to determine the consistency of the design.

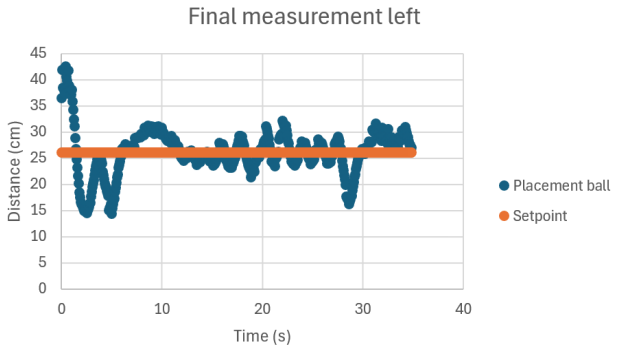


Figure 18: Final measurement left

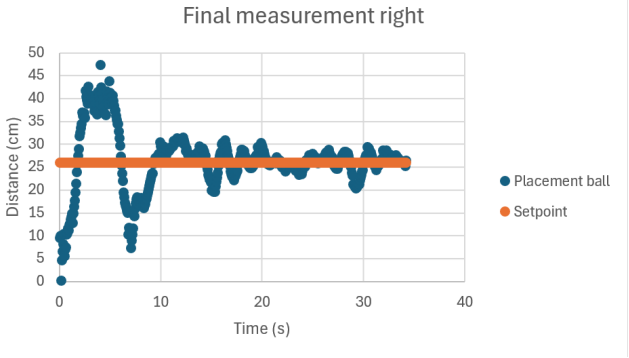


Figure 19: Final measurement right

The rising time is 1.4 s for the measurement with the ball on the left and 1.7 s for the ball on the right. A settling time can technically not be determined since the ball deviates more than a percent from a final value. There is no significant steady state error visible in the final measurements. That is logical, because the position of the ball is controlled by a PID controller. The integral component of the controller takes away the steady state error. The maximum overshoot is 11.5 cm for the measurement with the ball on the left and 21.5 cm with the ball on the right.

The rising time is acceptable to satisfy the earlier defined objective. However, the overshoot is quite large, especially when the ball is placed on the right side of the beam. Moreover, the system takes to long to "stabilize" the ball. The asymmetric measurements could come from a different motor speed when the motor is going left and right. This is also seen when looking at the working seesaw, because the seesaw seems to be more aggressive when turning clockwise, making the overshoot bigger when the ball needs to go to the left. Because of these factors, the objective has not been reached.

A disclaimer for these measurements is needed. The placement of the ball was measured using the distance sensors, but these sensors are far from ideal. Actually, the sensors are probably one of the reasons why the model deviates that much from the reality. Therefore, it could be that the measurements are a bit off. For the analysis, this does not really matter, as the same distance sensors are used in the controller, but it could be that the position is a bit different in reality.

7.1 Comparison model and measurements

The model is shown in figure 9 and 10 and the measurements are shown in figure 18 and 19. In the table below, you can see the different PID values for both the model and the design.

Table 2: Table with PID values

Type	**Type controller**	Kp	τ_i	τ_d
Model angle	PD	35	0.3	15
Model ball	PID	0.1	99	10
Design angle	P	0.1	999	0
Design ball	PID	0.4	1.33	1.25

As shown, the values are quite different for the angle control. This is again because the code uses different definitions for the PID parameters. It was observed that for the angle control there needs to be no τ_d and a high τ_i, even though the model says otherwise. This is because the rotary encoder is quite good, so the steady state error of the angle control can be neglected. The motor also reacts a lot faster than the ball, that is why it is not necessary to compensate any poles with derivative control.

The control of the ball was a PD controller in the model and a PID controller in real life.This is because a steady state error was observed, making it necessary to add integral control.

7.2 Improvements

The system could still be improved in a few ways. First of all, the system would be improved if better distance sensors were available. For the control, it is really important to determine the placement of the ball, otherwise, the controller will react wrongly and that could make the placement of the ball even worse.

Secondly, there probably exists an even better combination of PID values. Due to time constraints, it was only possible to do a few sets of tests. If more time was available, the PID values could be tweaked more carefully. This means that the values are increased in smaller amounts and more different combinations are tried. For example, the values for the angle control of the design. If we do use a PD controller, the motor would operate smoother.

Also, the difference is motor throttle when the motor turn left and right can be improved by adding a constant to the side that seems to react less strongly than the other. This would cause the system to react the same way if it is turning left and when it is turning left.

Lastly, the system could probably improve if the code would be changed in such a way that the motor would react less strongly if the ball is close to the setpoint. This is already implemented in the code, as the target angle changes with the position of the ball. However, as visible in the graphs of the measurements, the ball sometimes almost stabilizes at the setpoint and is then moved away from the setpoint again. This is not intentional of course, but probably happens because of a slight time delay in the code. This is something that can be explored in the future to improve the system even further.

8 Conclusion

In conclusion, this essay shows the design process of a self balancing seesaw. The objective was to stabilize a ping pong ball in the center of the beam with a steady state error of 20 mm and within 5 seconds. This objective was not met and these results are discussed above in chapter 7. Although our system does not accurately preform withing the objective, it does eventually stabilize the ball. It also keeps the ball on the beam and preforms the task consistently. Lastly, the set-up is also user friendly and portable. Improvements can be made to enhance the system. These are presented in paragraph 7.2.

References

[1] Cornell University. *PID Controller*. [Online; accessed 10-Oct-2025]. 2025. URL: https://people.ece.cornell.edu/land/courses/ece4760/FinalProjects/s2012/fas57_nyp7/Site/pidcontroller.html.

[2] J.W.J. Verschuur and H. Wormeester. *Modelling, Analysis and Control of Dynamic Systems*. [Version 9.05]. 2025.

A Calibration distance sensors

In this section, the process of calibrating the distance sensors is discussed. The process consists of measuring the voltage output of the sensor by different positions of the ball. This position ranges from 2 cm to 50 cm in steps of 1 cm. The center of the ball is taken as the starting point of measuring. This test is done for both the distance sensors on either side of the beam. After that, the test results are plotted and a fit is executed. Using this fit, you can now determine the position of the ball by a given voltage output of the distance sensors.

In figure 20, the test results are plotted for both sensors with the error for a distance of 2 cm to 50 cm. The error is calculated internally using the standard deviation.

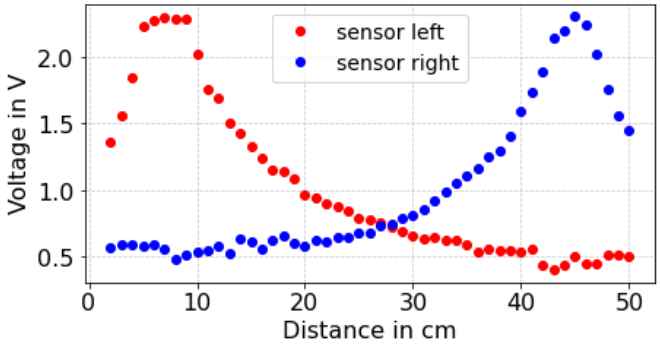


Figure 20: distance vs. voltage graph for both sensors

In figure 21 and 22, the test results are fitted. In this case only the detection range is plotted.

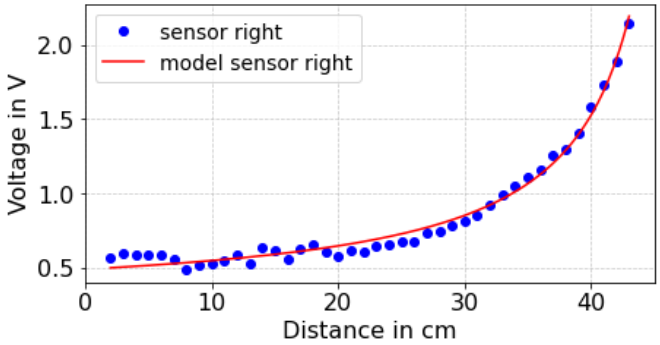


Figure 21: distance of ball w.r.t. the sensor vs. voltage graph for the right distance sensor

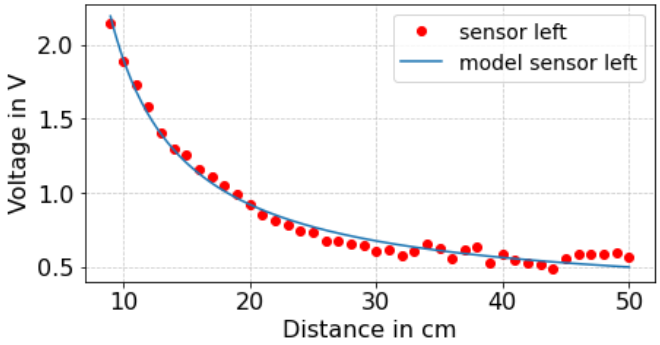


Figure 22: distance of ball w.r.t. the sensor vs. voltage graph for the left distance sensor

The fit has the distance of the ball w.r.t. the sensor as an input and the voltage of the sensor as output. In the model, the input and output needs to be reversed. This is done by inverting the formulas of the fits. These formulas are depicted below, with formula 17 for the left sensor and formula 18 for the right sensor.

$$distance = \frac{10.999}{voltage + 3.298} - 0.264 \tag{17}$$

$$distance = \frac{-10.998}{voltage + 1.239} + 0.264 + 50 \tag{18}$$

B Deriving of differential equation 5

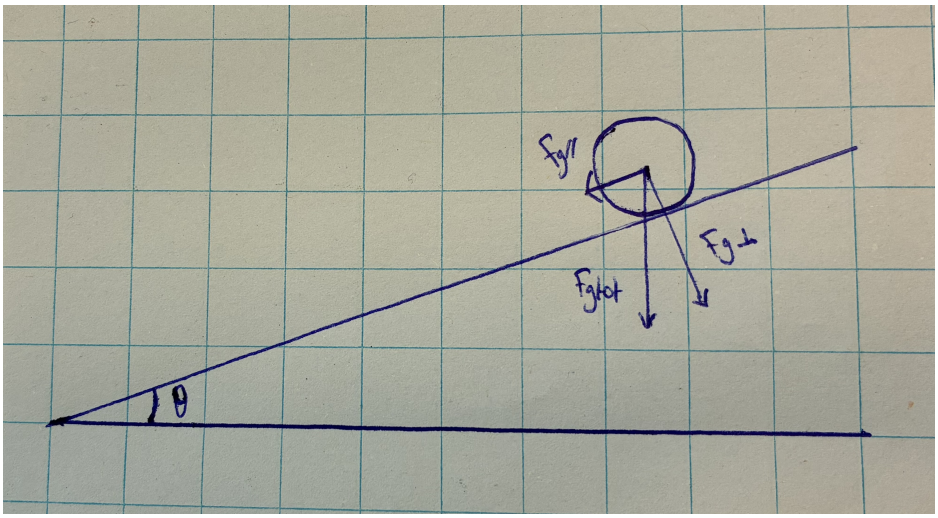


Figure 23: Ball on an incline

The total force on the ball by gravity is:

$$F_g = mg \tag{19}$$

Only the parallel component causes the ball to move on the incline:

$$F_g// = mgsin\theta \tag{20}$$

There is also rolling friction working on the ball. This is expressed as a torque, which can be written as a force as well:

$$\tau = I\alpha = RF \tag{21}$$

$$F_F = \frac{I\alpha}{R} \tag{22}$$

Finally, the inertia of a hollow sphere can be substituted in:

$$\frac{2}{3}mR^2 \tag{23}$$

$$F_F = I\frac{\alpha}{R} = \frac{2}{3}mR^2\frac{a}{R^2} = \frac{2}{3}ma \tag{24}$$

Finally, the final balance over the ball is:

$$F_r = F_g// - F_f = mgsin\theta - \frac{2}{3}ma = ma \tag{25}$$

$$\frac{5}{3}ma = mgsin\theta \tag{26}$$

$$ma = \frac{3}{5}mgsin\theta \tag{27}$$

This can be rewritten to the final differential equation, which is:

$$m\frac{dv}{dt} = \frac{3}{5}mgsin\theta \tag{28}$$

C Deriving of differential equation 2

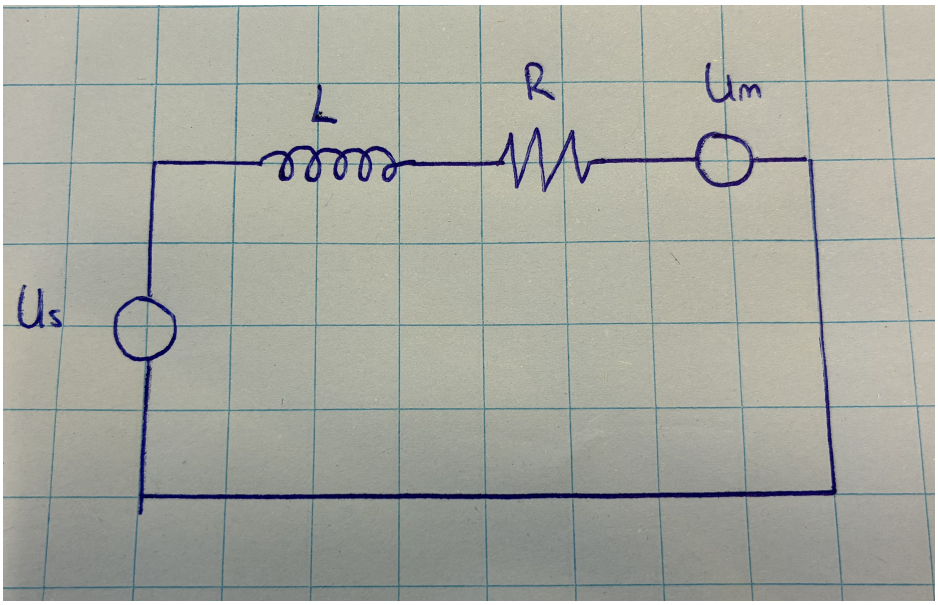


Figure 24: Circuit in electromotor

This is a typical circuit of an electromotor. U_s is the supplied voltage, L is a coil which transduces electrical energy into rotational energy, R is a resistor and U_m is the back-emf generated by the motor. The characteristic equation for a coil is:

$$L\frac{di}{dt} = U_L \tag{29}$$

Using Kirchhoff's law, the following relation for the voltages can be formulated:

$$U_s - U_L - U_r - U_m = 0 \tag{30}$$

If the differential for the coil is substituted in and the terms are rearranged, the following formula is derived:

$$L\frac{di}{dt} = U_s - Ri - Um \tag{31}$$

This is the final differential equation that will be used for the model.

D Code

```
#---Importing all packages
from simple_pid import PID
import time
import board
import busio
import rotaryio
import adafruit_ads1x15.ads1115 as ADS
from adafruit_ads1x15.analog_in import AnalogIn
from adafruit_motorkit import MotorKit
from digitalio import DigitalInOut, Direction, Pull
from adafruit_debouncer import Button

#---Hardware setup
pin1 = DigitalInOut(board.D12)
pin1.direction = Direction.INPUT
pin1.pull = Pull.UP
button1 = Button(pin1)
start = False

pin2 = DigitalInOut(board.D11)
pin2.direction = Direction.INPUT
pin2.pull = Pull.UP
button2 = Button(pin2)

i2c = busio.I2C(board.SCL, board.SDA)
kit = MotorKit(i2c=i2c)
ads = ADS.ADS1115(i2c)
encoder = rotaryio.IncrementalEncoder(board.D9, board.D10)

kit.motor2.throttle = 0
print("Code started")

#---Sensor settings
SENSOR_LEFT_PIN = ADS.P0
SENSOR_RIGHT_PIN = ADS.P1

ALPHA = 0.15
filtered_voltage_left = None
filtered_voltage_right = None

GAIN_VALUES = [6.144, 4.096, 2.048, 1.024, 0.512, 0.256]
GAIN_SETTINGS = [2/3, 1, 2, 4, 8, 16]

def apply_low_pass_filter(new_voltage, previous_filtered_voltage, alpha):
    if previous_filtered_voltage is None:
        return new_voltage
    return alpha * new_voltage + (1 - alpha) * previous_filtered_voltage

def auto_gain_read(channel):
    last_voltage = None
    for gain_index in range(len(GAIN_SETTINGS) - 1, -1, -1):
        ads.gain = GAIN_SETTINGS[gain_index]
        analog_in = AnalogIn(ads, channel)
        voltage = analog_in.voltage
        last_voltage = voltage
        max_voltage = GAIN_VALUES[gain_index]
        if abs(voltage) <= max_voltage * 0.9:
            return voltage
    return last_voltage
```

```
ads.gain = 1
time.sleep(1)

#---PID Controllers
pid_distance = PID(Kp=0.4, Ki=0.2, Kd=0.5, setpoint=26)
pid_distance.output_limits = (-5, 5)
pid_distance.sample_time = 0.005

pid_angle = PID(Kp=0.1, Ki=0, Kd=0, setpoint=0)
pid_angle.output_limits = (-1, 1)
pid_angle.sample_time = 0.001

#---Distance calibration constants
a1, b1, c1 = 10.999, -3.298, 0.264
a2, b2, c2 = 10.998, -1.239, 0.264
L = 50
EPS = 1e-3
MAX_PLAUSIBLE = 53

#--- Safe distance functions, no really small denominators
def safe_distance_left(v_left):
    if v_left is None:
        return None
    denom = v_left - c1
    if abs(denom) < EPS:
        return None
    d = abs((a1 / denom) - b1)
    return d if d < MAX_PLAUSIBLE else None

def safe_distance_right(v_right):
    if v_right is None:
        return None
    denom = v_right - c2
    if abs(denom) < EPS:
        return None
    d = abs((-a2 / denom) + b2 + L)
    return d if d < MAX_PLAUSIBLE else None

def final_distance_safe(v_left, v_right):
    l = safe_distance_left(v_left)
    r = safe_distance_right(v_right)

    if l is None and r is None:
        return None
    if l is None:
        return r
    if r is None:
        return l

#---Adding an extra offset to correct distance at the ends
    ave = (l + r) / 2
    if ave < 10:
        out = abs(r - 3)
    elif ave < 14:
        w = (ave - 10) / 4
        out = abs(w * ave + (1 - w) * (r - 3))
    elif ave > 40:
        out = abs(l + 3)
    elif ave > 36:
        w = (ave - 36) / 4
        out = abs(w * (l + 3) + (1 - w) * ave)
```

```
    else:
        out = abs(ave)

    return out if out < MAX_PLAUSIBLE else None

#---Motor throttle settings
min_throttle = 0.6
last_valid_distance = 26

#--- Main loop
while True:
    button1.update()
    button2.update()

    if button1.pressed:
        start = True
        start_time = time.monotonic()
        print("Logging gestart")

    if button2.pressed:
        start = False
        kit.motor2.throttle = 0
        print("Logging gestopt")

    if start:
        voltage_left = auto_gain_read(SENSOR_LEFT_PIN)
        voltage_right = auto_gain_read(SENSOR_RIGHT_PIN)

        filtered_voltage_left = apply_low_pass_filter(voltage_left,
            filtered_voltage_left, ALPHA)
        filtered_voltage_right = apply_low_pass_filter(voltage_right,
            filtered_voltage_right, ALPHA)

        final_afstand = final_distance_safe(filtered_voltage_left,
            filtered_voltage_right)

        #--- Use last valid measurement if measurement unvalid
        if final_afstand is None:
            final_afstand = last_valid_distance
        else:
            last_valid_distance = final_afstand

        # --- Cascade PID ---
        target_angle = pid_distance(final_afstand)
        pid_angle.setpoint = target_angle

        current_angle = encoder.position / 2.95
        motorspeed = pid_angle(current_angle)

        # --- Throttle mapping ---
        if abs(motorspeed) < 0.01:
            motorspeed_final = 0
        else:
            motorspeed_final = (min_throttle if motorspeed > 0 else -min_throttle) +
                (0.68 - min_throttle) * motorspeed

        kit.motor2.throttle = motorspeed_final

        # --- Debug output for Mu Plotter ---
        print(final_afstand)

    time.sleep(0.001)
```